FRONTEND ROADMAP

DAY 8

TypeScript Fundamentals I

JavaScript with a safety net. Learn why TypeScript exists, why it's a superset of JavaScript — not a new language — and write your first typed code.

WHAT WE COVER

- Why TypeScript
- Superset of JS
- How it compiles
- Setup with tsc
- Basic types
- Functions & unions

You already know JavaScript. TypeScript is that same language with one powerful addition: types that catch mistakes before your code ever runs. Today we'll see exactly why plain JavaScript lets bugs slip through, understand that TypeScript is a superset you can adopt gradually, set it up alongside the Node tools from Day 7, and learn the core types you'll use every day. Lots of small examples — try to predict each result.

TypeScript Fundamentals I

TypeScript is **JavaScript + types**. Every JavaScript file you've written is already valid TypeScript — we're not learning a new language, we're adding a safety layer on top of one you know. That layer catches typos and wrong types *as you type*, instead of at runtime in front of a user.

CLASS AGENDA · ~2 HOURS

0:00	Warm-up: where plain JavaScript lets bugs through	§1
0:15	What TypeScript is: a superset, and how it compiles	§2–3
0:40	Install & run your first TypeScript program	§4–5
1:05	Core types & type inference	§6–7
1:25	any vs unknown, then typing functions	§8–9
1:45	Unions, literals & wrap-up	§10

ON THIS HANDOUT

PART 1 · Why TypeScript?

01 The problem with plain JavaScript

JavaScript is **dynamically typed**: a variable can hold anything, and types are only checked when the code actually runs. That flexibility is also its weakness — many mistakes stay invisible until the program is running, often in front of a user.

A typo that says nothing

```js
const user = { name: "Abhishek" };
console.log(user.nmae);   // undefined -- typed "nmae", got no warning at all
```

A crash waiting to happen

```JS
function greet(name) {
  return "Hi " + name.toUpperCase();
}

greet();   // runtime crash: Cannot read properties of undefined (reading 'toUpperCase')
```

Nothing warned you while writing this. You only find out by running the code and hitting the error. In a large app these mistakes multiply, and you catch them by clicking around — not before shipping.

- Wrong number of arguments? No warning.
- Misspelled a property? You get `undefined`, silently.
- Passed a string where a number was expected? It quietly does the wrong thing.

02 TypeScript: a superset of JS

Here's the key idea for today: **TypeScript is a superset of JavaScript**. That means all JavaScript is valid TypeScript. You are not learning a second language — you're adding an optional **type layer** on top of the one you know.

- Every `.js` file is already valid TypeScript — rename it to `.ts` and it still works.
- You add small **type annotations** that describe what a value should be.
- TypeScript then checks those types **as you type**, underlining mistakes in your editor.

A simple way to picture it: JavaScript is a car; TypeScript is the same car wearing a seatbelt. Same engine, same driving — just far safer when something goes wrong.

```TS
// This is BOTH valid JavaScript and valid TypeScript:
let city = "Delhi";
console.log(city.toUpperCase());

// TypeScript just lets you ADD types when you want them:
let country: string = "India";
```

Because it's a superset, you can adopt it gradually — add types to the parts that matter and leave the rest as plain JS while you learn.

03 How TypeScript works (it compiles)

Browsers and Node don't run TypeScript directly — they only understand JavaScript. So TypeScript is **compiled** (transpiled) down to plain JavaScript by a tool called `tsc` (the TypeScript compiler).

- Step 1: you write `.ts` files with type annotations.
- Step 2: `tsc` checks the types, then **removes them**, producing normal `.js`.
- Step 3: you run that `.js` with Node or the browser — exactly like before.

Crucially, **types disappear at runtime**. They exist only while you develop, to catch mistakes. They add zero cost to the final program.

```ts
// what you write (add.ts)
function add(a: number, b: number): number {
  return a + b;
}

// what tsc produces (add.js) -- the types are simply gone
function add(a, b) {
  return a + b;
}
```

So the annotations are like guard-rails during construction: they keep you on track while building, then step out of the way of the finished product.

PART 2 · Setup & First Steps

04 Setting up TypeScript

TypeScript is just an npm package — so everything from Day 7 applies. Set it up inside a project folder:

```terminal
npm init -y                # create package.json (Day 7!)
npm install -D typescript  # add the TypeScript compiler as a dev dependency
npx tsc --init             # create tsconfig.json (the compiler's settings)
```

The `tsconfig.json` controls how your code is compiled. A few settings you'll meet early:

- `target` — which JavaScript version to output (e.g. `ES2020`).
- `outDir` — the folder for the compiled `.js` files (e.g. `dist`).
- `strict` — turns on all the safety checks. Keep it `true`.

Compile & run

```terminal
npx tsc           # compiles your .ts files into .js
node dist/index.js # run the compiled JavaScript
```

A faster loop while learning

Compiling then running is two steps. The `tsx` package runs a `.ts` file directly in one step — great for practice:

```terminal
npm install -D tsx
npx tsx index.ts   # type-checks + runs in one command
```

TRY IT YOURSELF

Create a folder, run the three setup commands, then compile an empty `index.ts` with `npx tsc` and confirm a `.js` file appears.

05 Your first typed program

Create `index.ts` and add a type annotation with a colon:

```ts
let message: string = "Hello, TypeScript!";
console.log(message);

message = 42;   // Error: Type 'number' is not assignable to type 'string'.
```

That last line is the whole point: TypeScript underlines it in **red in your editor, before you run anything**. In plain JavaScript this would silently run and cause a bug later. Here, the mistake is caught immediately.

- The annotation `: string` tells TS what `message` is allowed to hold.
- Assigning a number breaks that promise, so TS complains right away.

PART 3 · Core Types

06 Basic types

The everyday types you'll annotate with:

```ts
let title: string = "Frontend Roadmap";
let count: number = 42;          // ints and decimals are both "number"
let isDone: boolean = false;

let scores: number[] = [90, 85, 100];   // an array of numbers
let names: string[] = ["Abhi", "Sam"];  // an array of strings

let pair: [string, number] = ["age", 25];   // a tuple: fixed length + types
```

- `string`, `number`, `boolean` cover most simple values.
- Arrays: `number[]` means “array of numbers”.
- A `tuple` is an array with a fixed length and a type per position.

07 Type inference: let TS figure it out

You don't have to annotate everything. When you give a value straight away, TypeScript **infers** the type for you.

```ts
let count = 0;          // TS infers: number
let title = "Roadmap";  // TS infers: string

count = 5;      // fine
count = "five"; // Error: string is not assignable to number
```

- Let TS infer simple variables — cleaner code, same safety.
- Do annotate **function parameters** (TS can't guess those) and anywhere the type isn't obvious.

Rule of thumb: annotate the *edges* of your code (function inputs/outputs), let inference handle the middle.

08 any, unknown & why any is a trap

`any` turns type-checking **off** for a value — it can be anything and TS stops protecting you. It's tempting but defeats the purpose.

```ts
let data: any = "hello";
data.toUpperCase();   // ok
data.doSomething();   // TS allows it -> crashes at runtime. any = no safety net.
```

When you truly don't know a type yet, use `unknown` instead. It's the safe version: TS forces you to check what it is before using it.

```ts
let value: unknown = getInput();

// must narrow it first:
if (typeof value === "string") {
  console.log(value.toUpperCase());   // safe here
}
```

- Avoid `any` — it silently switches off TypeScript.
- Prefer `unknown` + a check when a type is genuinely uncertain.

09 Typing functions

Functions are where types pay off most — you declare what goes in and what comes out.

```ts
function greet(name: string): string {
  return "Hi " + name;
}

function logMessage(msg: string): void {   // void = returns nothing
  console.log(msg);
}

// optional (?) and default parameters
function power(base: number, exp: number = 2): number {
  return base ** exp;
}

power(5);     // 25  (exp defaults to 2)
power(2, 3);  // 8
```

- Each parameter gets a type; the type after the `)` is the return type.
- `void` means the function doesn't return a value.
- A `?` makes a parameter optional; `= value` gives it a default.

> **TRY IT YOURSELF**
>
> Write a typed function `rectangleArea(width: number, height: number)` that returns a `number`. Call it and log the result.

10 Union & literal types

Sometimes a value can be one of several types. A **union** (`|`) says "this OR that".

```ts
function formatId(id: string | number): string {
  return "ID-" + id;
}

formatId(7);      // "ID-7"
formatId("abc");  // "ID-abc"
```

A **literal type** restricts a value to specific allowed options — perfect for things like a status or a direction:

```ts
type Direction = "up" | "down" | "left" | "right";

function move(dir: Direction): string {
  return "moving " + dir;
}

move("up");     // fine
move("north");  // Error: "north" is not a valid Direction
```

Literal unions are everywhere in real apps and you'll use them constantly in React (for example a request state of `loading | success | error`). We'll lean on them tomorrow.

> **TRY IT YOURSELF**
>
> Create a type `Size = "small" | "medium" | "large"` and a function `price(size: Size)` that returns a different number for each.

Practice Challenges

Try a few — they use everything from today:

- Retype the Day 7 calculator's `calculate` function so `a` and `b` are numbers.
- Write `fullName(first: string, last: string): string` and log a few names.
- Make a `type Grade = "A" | "B" | "C"` and a function that maps a score to a grade.
- Take any small `.js` script you wrote before, rename it `.ts`, and add types until `tsc` is happy.

Conclusion & what's next

You now understand *why* TypeScript exists: plain JavaScript hides type mistakes until runtime, and TypeScript — a **superset** that compiles down to JavaScript — catches them while you write. You set it up, wrote typed variables and functions, and met unions and literal types.

Next class (Day 9): we model real data with **objects, interfaces and generics**, then convert the Day 7 terminal calculator into TypeScript — the exact skills you'll use to type React components on Day 10.

> **QUICK RECAP**
>
> TypeScript = **JavaScript + types**, a **superset** (all JS is valid TS) · it **compiles** to plain JS and types vanish at runtime · set it up with **npm + tsc** · annotate function inputs/outputs, let inference do the rest · avoid **any**, and use **unions** & **literals** for flexible, safe values.